

08 Three.js Snippets.md

08 Three.js Snippets

Runnable Three.js demos and copy-paste JS for **IGWT** Course 8 (and lights/PBR bits of 11).

Do this **after** a student can draw a triangle in 07 WebGL and Shader Snippets. Three.js hides the pipeline; keep asking “what is this in GL?”

Start

1. 00 Index
2. Open `ThreeJS/demos/01-hello-cube.html` (needs network once: CDN)
3. 01 Scene Camera Renderer

Import map (every demo):

```
<script type="importmap">
{
  "imports": {
    "three": "https://cdn.jsdelivr.net/npm/three@0.170.0/build/three.module.js",
    "three/addons/": "https://cdn.jsdelivr.net/npm/three@0.170.0/examples/jsm/"
  }
}
</script>
```

Pinned **r170**. If the CDN is blocked, download the same files into `ThreeJS/vendor/` and point the import map at them.

What is in the folder

Path	What it is
00 Index	Demo map
01 Scene Camera Renderer	Canvas, camera, loop, resize, color space
02 Lights and Shadows	Lights, shadow maps
03 Materials and Textures	Standard/physical, canvas textures
04 Scene Graph and Animation	Groups, Clock, solar-system pattern
05 Picking	Raycaster, layers
06 Loaders	glTF, texture, dispose
07 ShaderMaterial	Raw/ShaderMaterial, uniforms
08 Performance	Instancing, frustum, LOD

Path	What it is
09 Postprocess	Composer, bloom
10 Debug	Helpers, HUD, “what is this in GL?”
ThreeJS/demos/	One-idea HTML files
ThreeJS/js/	Copy-paste modules (read in an editor; demos stay self-contained for <code>file://</code>)

Teaching use

- Week 1: demos 01–04, 01 Scene Camera Renderer
- Week 2: 05–07, lights and materials
- Week 3: 08–10, picking + shader
- Week 4: 11–15, loader, particles, env, bloom
- Always: 10 Debug

License

Teaching notes. Reuse with attribution.

00 Index.md

Three.js snippet index

Parent: 08 Three.js Snippets

Open `ThreeJS/demos/`. Each file is `type="module"` and loads **three r170** from jsDelivr. Needs the network the first time.

Demos

File	Idea
<code>demos/01-hello-cube.html</code>	Scene, PerspectiveCamera, WebGLRenderer, Mesh
<code>demos/02-orbit.html</code>	OrbitControls
<code>demos/03-lights-shadows.html</code>	Directional light + shadow map + plane
<code>demos/04-materials.html</code>	Basic / Lambert / Phong / Standard / Physical
<code>demos/05-canvas-texture.html</code>	CanvasTexture, no CORS
<code>demos/06-solar-system.html</code>	Groups, nested transforms, Clock
<code>demos/07-raycaster.html</code>	Click to pick, highlight
<code>demos/08-instancing.html</code>	InstancedMesh
<code>demos/09-shader-material.html</code>	ShaderMaterial + time uniform
<code>demos/10-gltf-pattern.html</code>	GLTFLoader pattern (procedural stand-in + real load snippet)
<code>demos/11-particles.html</code>	Points + BufferGeometry
<code>demos/12-fog-helpers.html</code>	Fog, GridHelper, AxesHelper
<code>demos/13-environment.html</code>	Hemisphere + RoomEnvironment-style IBL-lite
<code>demos/14-resize.html</code>	Pixel ratio, canvas CSS vs drawing buffer
<code>demos/15-lod.html</code>	LOD three levels
<code>demos/16-custom-buffer.html</code>	BufferGeometry from arrays (comp-geom bridge)
<code>demos/17-tone-mapping.html</code>	ACES + outputColorSpace
<code>demos/18-bloom.html</code>	EffectComposer + UnrealBloomPass
<code>demos/19-sprites-labels.html</code>	Sprite / canvas label
<code>demos/20-shadow-contact.html</code>	PCF shadows, bias

Catalogs

Note	Contents
01 Scene Camera Renderer	Boot sequence
02 Lights and Shadows	Light types, shadow knobs
03 Materials and Textures	PBR fields, color space
04 Scene Graph and Animation	Groups, quaternions, mixer
05 Picking	Raycaster
06 Loaders	glTF checklist
07 ShaderMaterial	GLSL in Three
08 Performance	Draw calls, dispose
09 Postprocess	Composer
10 Debug	Helpers, GL mapping

How to use in a lab

1. Open one demo.
2. Change one number (camera fov, metalness, instance count).
3. Ask: world / view / clip — which space did you just change?
4. Copy a snippet from a catalog into a blank HTML with the import map from 08 Three.js Snippets.

01 Scene Camera Renderer.md

01 — Scene, camera, renderer

Parent: 08 Three.js Snippets

Demo: [demos/01-hello-cube.html](#), [demos/14-resize.html](#).

Minimum boot

```
import * as THREE from "three";

const canvas = document.getElementById("c");
const renderer = new THREE.WebGLRenderer({ canvas, antialias: true });
renderer.setPixelRatio(Math.min(devicePixelRatio, 2));
renderer.setSize(canvas.clientWidth, canvas.clientHeight, false);
renderer.outputColorSpace = THREE.SRGBColorSpace;

const scene = new THREE.Scene();
scene.background = new THREE.Color(0x1a1a1e);

const camera = new THREE.PerspectiveCamera(50, canvas.clientWidth / canvas.clientHeight, 0.1, 100);
camera.position.set(2.4, 1.6, 3.2);
camera.lookAt(0, 0, 0);

const geo = new THREE.BoxGeometry(1, 1, 1);
const mat = new THREE.MeshStandardMaterial({ color: 0xe85d3a, roughness: 0.45, metalness: 0.05 });
scene.add(new THREE.Mesh(geo, mat));
scene.add(new THREE.HemisphereLight(0xffffffff, 0x334455, 1.1));

function frame(t) {
  renderer.render(scene, camera);
  requestAnimationFrame(frame);
}
requestAnimationFrame(frame);
```

Resize (always)

```
function resize() {
  const w = canvas.clientWidth, h = canvas.clientHeight;
  const dpr = Math.min(devicePixelRatio, 2);
  if (canvas.width !== Math.floor(w * dpr) || canvas.height !== Math.floor(h * dpr)) {
    renderer.setPixelRatio(dpr);
    renderer.setSize(w, h, false);
    camera.aspect = w / h;
    camera.updateProjectionMatrix();
  }
}
```

Call `resize()` at the start of `frame`. Forgetting `updateProjectionMatrix` stretches the cube.

Cameras

```
const persp = new THREE.PerspectiveCamera(50, aspect, 0.1, 100);
const ortho = new THREE.OrthographicCamera(-2 * aspect, 2 * aspect, 2, -2, 0.1, 100);
```

Near `0.1`, far `100` is the teaching default. Near `0.001` causes z-fighting.

Color management (r152+)

```
renderer.outputColorSpace = THREE.SRGBColorSpace;
renderer.toneMapping = THREE.ACESFilmicToneMapping;
renderer.toneMappingExposure = 1.0;
// MeshStandardMaterial.color is sRGB. Texture.colorSpace = THREE.SRGBColorSpace for albedo.
```

Do **not** set `texture.colorSpace = SRGB` on a linear data map (roughness, metalness, normal).

Orthographic HUD camera (optional)

```
const hudCam = new THREE.OrthographicCamera(-1, 1, 1, -1, 0, 1);
```

What this is in GL

Three.js	WebGL
<code>scene + matrixWorld</code>	model matrix
<code>camera.matrixWorldInverse</code>	view matrix
<code>camera.projectionMatrix</code>	projection
<code>renderer.render</code>	clear, bind program, draw

Three.js	WebGL
outputColorSpace	last gamma / framebuffer

02 Lights and Shadows.md

02 — Lights and shadows

Parent: 08 Three.js Snippets

Demos: [demos/03-lights-shadows.html](#), [demos/20-shadow-contact.html](#).

Light types (teaching set)

```
scene.add(new THREE.AmbientLight(0xffffff, 0.15));
scene.add(new THREE.HemisphereLight(0x9bb7ff, 0x3a2a1a, 0.45));

const sun = new THREE.DirectionalLight(0xfff2dd, 1.6);
sun.position.set(4, 6, 3);
sun.castShadow = true;
scene.add(sun);

const bulb = new THREE.PointLight(0xff6622, 8, 12, 2); // intensity, distance, decay
bulb.position.set(-1, 2, 1);
scene.add(bulb);

const spot = new THREE.SpotLight(0xffffff, 6, 20, Math.PI / 7, 0.4, 1);
spot.position.set(2, 5, 2);
spot.target.position.set(0, 0, 0);
scene.add(spot);
scene.add(spot.target);
```

`RectAreaLight` needs `RectAreaLightUniformsLib.init()` and only works with Standard/Physical.

Shadow map (directional)

```
renderer.shadowMap.enabled = true;
renderer.shadowMap.type = THREE.PCFSoftShadowMap;

sun.castShadow = true;
sun.shadow.mapSize.set(1024, 1024);
sun.shadow.camera.near = 0.5;
sun.shadow.camera.far = 20;
sun.shadow.camera.left = -6;
sun.shadow.camera.right = 6;
sun.shadow.camera.top = 6;
sun.shadow.camera.bottom = -6;
sun.shadow.bias = -0.0004;
sun.shadow.normalBias = 0.02;

mesh.castShadow = true;
mesh.receiveShadow = true;
ground.receiveShadow = true;
```

Acne → more `bias` / `normalBias`. Peter-panning → less bias. Peter-panning + acne together → shadow camera too tight or map too small.

Debug the shadow camera

```
scene.add(new THREE.CameraHelper(sun.shadow.camera));
```

If the helper does not contain the mesh, the shadow will clip.

What this is in GL

A directional shadow is demo [WebGL/demos/22-shadow-map.html](https://www.khronos.org/webgl/demos/22-shadow-map.html): light-space MVP, depth compare, bias. Three.js writes that pass for you. Students should still say “bias” and “map size” out loud.

03 Materials and Textures.md

03 — Materials and textures

Parent: 08 Three.js Snippets

Demos: [demos/04-materials.html](#), [demos/05-canvas-texture.html](#), [demos/17-tone-mapping.html](#).

Which material

Class	Lighting	Use
MeshBasicMaterial	none	unlit, debug UV, sprites
MeshLambertMaterial	Lambert	cheap, no spec
MeshPhongMaterial	Blinn-Phong	shiny plastic (old demos)
MeshStandardMaterial	PBR metal/rough	default for the course
MeshPhysicalMaterial	PBR + clearcoat / transmission	glass, car paint
MeshNormalMaterial	none	debug normals
MeshDepthMaterial	none	debug depth

```
const std = new THREE.MeshStandardMaterial({
  color: 0xc45c38,
  roughness: 0.35,
  metalness: 0.0,
  envMapIntensity: 1.0,
});
```

Canvas texture (no CORS)

```
const cnv = document.createElement("canvas");
cnv.width = cnv.height = 64;
const ctx = cnv.getContext("2d");
for (let y = 0; y < 8; y++) for (let x = 0; x < 8; x++) {
  ctx.fillStyle = (x + y) % 2 ? "#2850b4" : "#f0f0f0";
  ctx.fillRect(x * 8, y * 8, 8, 8);
}
const tex = new THREE.CanvasTexture(cnv);
tex.colorSpace = THREE.SRGBColorSpace;
tex.wrapS = tex.wrapT = THREE.RepeatWrapping;
tex.repeat.set(2, 2);
tex.anisotropy = renderer.capabilities.getMaxAnisotropy();
```

After drawing into an existing canvas: `tex.needsUpdate = true`.

Color space

```
albedo.colorSpace = THREE.SRGBColorSpace;  
roughnessMap.colorSpace = THREE.NoColorSpace; // linear data  
normalMap.colorSpace = THREE.NoColorSpace;
```

Physical extras

```
const glass = new THREE.MeshPhysicalMaterial({  
  color: 0xffffffff,  
  transmission: 0.9,  
  thickness: 0.4,  
  roughness: 0.05,  
  ior: 1.5,  
  attenuationColor: 0x88ccff,  
  attenuationDistance: 1.2,  
});
```

Transmission is expensive. One glass object in a student project is enough.

Shared materials

If two meshes share a material, changing `mesh.material.color` changes both. Clone when you need a unique tint:

```
mesh.material = mesh.material.clone();  
mesh.material.color.set(0x33aa66);
```

04 Scene Graph and Animation.md

04 — Scene graph and animation

Parent: 08 Three.js Snippets

Demo: [demos/06-solar-system.html](#).

Groups (the point of Three.js)

```
const solar = new THREE.Group();
const earthPivot = new THREE.Group();
const moonPivot = new THREE.Group();
earthPivot.add(earth);
moonPivot.add(moon);
earth.add(moonPivot);
solar.add(sun, earthPivot);
scene.add(solar);

earthPivot.position.x = 3;
moonPivot.position.x = 0.8;

const clock = new THREE.Clock();
function frame() {
  const dt = clock.getDelta();
  const t = clock.getElapsedTime();
  earthPivot.rotation.y += dt * 0.4;
  moonPivot.rotation.y += dt * 1.6;
  earth.rotation.y = t * 1.2;
  renderer.render(scene, camera);
  requestAnimationFrame(frame);
}
```

A mesh's `position` is **relative to its parent**. That is the whole lecture.

Transforms

```
mesh.position.set(1, 0, 0);
mesh.rotation.order = "YXZ"; // or Quaternion
mesh.quaternion.setFromAxisAngle(new THREE.Vector3(0, 1, 0), Math.PI / 4);
mesh.scale.setScalar(1.5);
mesh.updateMatrixWorld();
```

Prefer quaternions for interpolation. Euler yaw/pitch is fine for orbit.

Look-at

```
mesh.lookAt(target.position);  
camera.lookAt(0, 0, 0);
```

`lookAt` changes rotation. Do not mix with a custom quaternion the same frame unless you mean to.

AnimationMixer (glTF clips)

```
const mixer = new THREE.AnimationMixer(model);  
const action = mixer.clipAction(gltf.animations[0]);  
action.play();  
function frame() {  
  mixer.update(clock.getDelta());  
}
```

No clips → no mixer. Do not add an empty mixer "just in case."

Object3D traversal

```
model.traverse((o) => {  
  if (o.isMesh) {  
    o.castShadow = true;  
    o.receiveShadow = true;  
  }  
});
```

05 Picking.md

05 — Picking (raycaster)

Parent: 08 Three.js Snippets

Demo: [demos/07-raycaster.html](#).

Pointer to NDC

```
const raycaster = new THREE.Raycaster();
const pointer = new THREE.Vector2();

function ndc(event) {
  const r = canvas.getBoundingClientRect();
  pointer.x = ((event.clientX - r.left) / r.width) * 2 - 1;
  pointer.y = -((event.clientY - r.top) / r.height) * 2 + 1;
}

canvas.addEventListener("pointerdown", (e) => {
  ndc(e);
  raycaster.setFromCamera(pointer, camera);
  const hits = raycaster.intersectObjects(pickables, false);
  if (hits.length) {
    const mesh = hits[0].object;
    mesh.material.emissive?.setHex(0x333333);
  }
});
```

y is flipped. Forgetting the minus sign picks the floor.

recursive true if the pickable is a Group.

Layers

```
camera.layers.enable(1);
ghost.layers.set(1); // only visible / pickable with that mask
raycaster.layers.set(0); // ignore ghosts
```

GPU picking (later)

For dense instancing, a raycaster on CPU is $O(n)$. Teach CPU picking first. Instanced picking: `InstancedMesh.raycast` works in current three; still cap instance count in class.

What this is in GL

A ray in world space vs triangles. Same idea as BVH picking in 05 Sample Graduation Thesis. Three.js uses a BVH only if you add one; default is brute force.

06 Loaders.md

06 — Loaders (glTF)

Parent: 08 Three.js Snippets

Demo: <demos/10-gltf-pattern.html>.

The load you will actually write

```
import { GLTFLoader } from "three/addons/loaders/GLTFLoader.js";
import { DRACOLoader } from "three/addons/loaders/DRACOLoader.js";

const draco = new DRACOLoader();
draco.setDecoderPath("https://www.gstatic.com/draco/v1/decoders/");
const loader = new GLTFLoader();
loader.setDRACOLoader(draco);

loader.load(
  "models/hero.glb",
  (gltf) => {
    const root = gltf.scene;
    root.traverse((o) => {
      if (o.isMesh) {
        o.castShadow = true;
        o.receiveShadow = true;
      }
    });
    scene.add(root);
  },
  (ev) => { /* percent = ev.loaded / ev.total */ },
  (err) => console.error(err)
);
```

Checklist (read with students)

- Export **glTF 2.0 / glb**, meters, +Y up
- Apply scale in Blender before export
- One texture set, not 8k extras
- `texture.colorSpace` on albedo after load if it looks too dark/bright
- `renderer.outputColorSpace = SRGBColorSpace`
- Shadows: `castShadow` on meshes, light configured
- Dispose the previous model if you reload

Texture loader

```
const tex = await new THREE.TextureLoader().loadAsync("albedo.jpg");
tex.colorSpace = THREE.SRGBColorSpace;
tex.wrapS = tex.wrapT = THREE.RepeatWrapping;
```

file:// plus a relative jpg often fails CORS. Use a static server or a CanvasTexture in week 1.

Dispose (memory)

```
function disposeObject(root) {
  root.traverse((o) => {
    if (o.geometry) o.geometry.dispose();
    const m = o.material;
    if (!m) return;
    const list = Array.isArray(m) ? m : [m];
    for (const mat of list) {
      for (const k of Object.keys(mat)) {
        const v = mat[k];
        if (v && v.isTexture) v.dispose();
      }
      mat.dispose();
    }
  });
}
```

Offline class

If the network is dead, the demo uses a torus-knot stand-in and still runs the **same** traverse / shadow / scale code you would run on `gltf.scene`.

07 ShaderMaterial.md

07 — ShaderMaterial

Parent: 08 Three.js Snippets

Demo: <demos/09-shader-material.html>. Raw GLSL lives in 07 WebGL and Shader Snippets.

ShaderMaterial (Three includes chunks)

```
const mat = new THREE.ShaderMaterial({
  uniforms: {
    u_time: { value: 0 },
    u_color: { value: new THREE.Color(0xe85d3a) },
  },
  vertexShader: /* glsl */ `
    varying vec3 v_n;
    varying vec2 v_uv;
    void main() {
      v_n = normalize(normalMatrix * normal);
      v_uv = uv;
      gl_Position = projectionMatrix * modelViewMatrix * vec4(position, 1.0);
    }
  `,
  fragmentShader: /* glsl */ `
    uniform float u_time;
    uniform vec3 u_color;
    varying vec3 v_n;
    varying vec2 v_uv;
    void main() {
      float l = max(dot(normalize(v_n), normalize(vec3(0.4, 1.0, 0.3))), 0.12);
      float bands = floor((v_uv.y + u_time * 0.1) * 8.0) / 8.0;
      gl_FragColor = vec4(u_color * l * (0.6 + 0.4 * bands), 1.0);
    }
  `,
});
```

Built-in: position, normal, uv, projectionMatrix, modelViewMatrix, normalMatrix.

Update:

```
mat.uniforms.u_time.value = clock.getElapsedTime();
```

RawShaderMaterial

No Three chunks, no automatic `#version`. You write WebGL2 GLSL yourself. Use this when bridging to Course 7.

```
const raw = new THREE.RawShaderMaterial({
  glslVersion: THREE.GLSL3,
  vertexShader: `#version 300 es
    in vec3 position;
    uniform mat4 modelViewMatrix, projectionMatrix;
    void main() {
      gl_Position = projectionMatrix * modelViewMatrix * vec4(position, 1.0);
    }`,
  fragmentShader: `#version 300 es
    precision highp float;
    out vec4 outColor;
    void main() { outColor = vec4(0.91, 0.31, 0.22, 1.0); }`,
});
```

OnBeforeCompile (patch Standard)

```
material.onBeforeCompile = (shader) => {
  shader.fragmentShader = shader.fragmentShader.replace(
    "#include <output_fragment>",
    `gl_FragColor.rgb = mix(gl_FragColor.rgb, vec3(1.0,0.4,0.1), 0.15);
    #include <output_fragment>`
  );
};
```

Fragile across three versions. Prefer ShaderMaterial for class.

Fire / water as a Three mesh

Put a Shadertoy-style `mainImage` on a **fullscreen triangle** or a plane in front of the camera. Snippets: 18 Shadertoy Effects. Do not wrap a 400-line Image shader on a box UV unless you meant that.

08 Performance.md

08 — Performance

Parent: 08 Three.js Snippets

Demos: [demos/08-instancing.html](#), [demos/15-lod.html](#).

InstancedMesh

```
const n = 400;
const mesh = new THREE.InstancedMesh(geo, mat, n);
const dummy = new THREE.Object3D();
const color = new THREE.Color();
for (let i = 0; i < n; i++) {
  dummy.position.set((i % 20) - 9.5, 0, Math.floor(i / 20) - 9.5);
  dummy.scale.setScalar(0.35);
  dummy.updateMatrix();
  mesh.setMatrixAt(i, dummy.matrix);
  mesh.setColorAt(i, color.setHSL(i / n, 0.55, 0.5));
}
mesh.instanceMatrix.needsUpdate = true;
if (mesh.instanceColor) mesh.instanceColor.needsUpdate = true;
scene.add(mesh);
```

One draw call. Changing one instance: `setMatrixAt` + `needsUpdate`.

Frustum culling

Default `mesh.frustumCulled = true`. For huge world matrices that confuse the AABB, set `frustumCulled = false` **or** fix `geometry.computeBoundingSphere()`.

LOD

```
const lod = new THREE.LOD();
lod.addLevel(high, 0);
lod.addLevel(mid, 8);
lod.addLevel(low, 18);
scene.add(lod);
```

Merge / reuse

- Same geometry + same material → one `InstancedMesh`, not 400 `Mesh`

- `texture.anisotropy` only as high as needed
- Cap pixel ratio at 2
- Shadows: 1024 is enough in class; 4096 is a laptop fan

Renderer info HUD

```
const { calls, triangles } = renderer.info.render;  
hud.textContent = calls + " draws, " + triangles + " tris";
```

`renderer.info.autoReset` is true. Read it after `render()`.

Dispose

See 06 Loaders. Leaking geometries is the usual "it got slow after 20 minutes" bug in labs.

09 Postprocess.md

09 — Postprocess

Parent: 08 Three.js Snippets

Demo: [demos/18-bloom.html](#).

Bloom (the one students want)

```
import { EffectComposer } from "three/addons/postprocessing/EffectComposer.js";
import { RenderPass } from "three/addons/postprocessing/RenderPass.js";
import { UnrealBloomPass } from "three/addons/postprocessing/UnrealBloomPass.js";
import { OutputPass } from "three/addons/postprocessing/OutputPass.js";

const composer = new EffectComposer(renderer);
composer.addPass(new RenderPass(scene, camera));
const bloom = new UnrealBloomPass(
  new THREE.Vector2(canvas.clientWidth, canvas.clientHeight),
  0.55, // strength
  0.4,  // radius
  0.85  // threshold
);
composer.addPass(bloom);
composer.addPass(new OutputPass());

function frame() {
  resize();
  composer.render();
}
```

On resize: `composer.setSize(w, h)`.

Threshold high → only emissive / bright lights bloom. Threshold 0 → a milky mess.

Cheap alternative

Skip the composer. Put a soft `Sprite` or additive `Points` around lights. Faster, less “Unreal.”

Order

Tone mapping belongs at the end (`OutputPass` in recent three). If the image is grey or over-bright, you applied ACES twice.

What this is in GL

Same idea as [WebGL/demos/13-framebuffer-post.html](#): scene FBO, fullscreen triangle, mix. Three.js is a stack of those passes.

10 Debug.md

10 — Debug and “what is this in GL?”

Parent: 08 Three.js Snippets

Demo: [demos/12-fog-helpers.html](#).

Helpers (leave them on in week 1)

```
scene.add(new THREE.AxesHelper(1.5));
scene.add(new THREE.GridHelper(10, 10, 0x444444, 0x2a2a2a));
scene.add(new THREE.DirectionalLightHelper(sun, 0.5));
scene.add(new THREE.CameraHelper(sun.shadow.camera));
```

`BoxHelper`, `PolarGridHelper`, `ArrowHelper` as needed.

Fog

```
scene.fog = new THREE.Fog(0x1a1a1e, 8, 22);
scene.background = scene.fog.color;
```

or `FogExp2(color, density)`.

Normal / UV debug materials

```
mesh.material = new THREE.MeshNormalMaterial();
// or
mesh.material = new THREE.MeshBasicMaterial({ map: uvGrid });
```

Log the program

```
console.log(renderer.capabilities.isWebGL2);
console.log(renderer.info);
```

Map to Course 7

Student sees	Make them say
Black mesh	lights? visible? camera looking? scale 0?
Mesh too dark	color space, no lights, metalness 1 without env

Student sees	Make them say
Shadows missing	<code>castShadow</code> / <code>receiveShadow</code> / <code>shadowMap.enabled</code> / light camera
Stretch on resize	<code>aspect</code> + <code>updateProjectionMatrix</code>
400 draw calls	should be <code>InstancedMesh</code>
Shadertoy on a cube	UVs vs <code>gl_FragCoord</code>

Lab question (every week)

Point at this object. Is the number you changed in **object**, **world**, **view**, or **clip** space?

If they cannot answer, they are using Three.js as a toy, not as a graphics course.